# Red-Teaming Large Language Model-Powered Recommendation Systems

Sbaaoui Idriss
Student ID: 22060004
Department / Major: Computer Science
University: 华东理工大学
Supervisor: 张恒润

**Abstract**

In recent years, we have started to see a surge of LLM or large language models being used in everyday life as a powerful personal helper. This approach helps millions with every day tasks, but it also starts replacing many automated task to be more AI driven. LLMs are at the core of many recommendation pipelines for retrieval, re-ranking and explanation generation. It thus shows a clear shift in the way we evaluate these recommender systems, as they are more prone to be manipulated by harmful actors. This proposal presents this core issue through our subject, *Red-Teaming Large Language Model-Powered Recommendation Systems*. With the use of my experimental project RAG Poison Lab, we will be able to track poisoning signals in every step of the retrieval process until the final recommendation output[1] using real life metrics to get a measurable outputs, HR@K, NDCG@K, MRR@K, ASR@K, candidate overlap, and target-rank lift. We are hoping to answer the following questions: where is the earliest step in which an actor can attack the system, and how much can they manipulate the final output. Our plan will run in a controlled environment where we will compare the baseline recommender results to the attacked ones simulating the real experience of an attacked RAG system.

# 1 Research Background

## 1.1 RAG-Based Recommendation Systems

The core of this thesis will be built around my own system, RAG-Poison-Lab [1], that englobes all possible fails and risky behavior an actor could affect rankings with from the retrieval to ranking to the final output. A user profile is converted to a query, candidates are retrieved from Elasticsearch, and top-$K$ items are ranked for output and trace inspection.

Those are the basic concepts of a retrieval-augmented generation (RAG) system: if retrieval changes, final model behavior also changes [2]. In a reccomender system, Large Language Models (LLMs) also take care of re-ranking, which opens a window for more risks since retrieved text can hurt the ranking [3]. To get the complete picture, I made sure that my system allows for both simple deterministic ranking and LLM re-ranking, which will help us see the result we get from both these recommendation methods.

We will use the dataset MovieLens 100K, a benchmarking dataset containing more than 100,000 rating of 9000 movies by almost 1000 users [4]. The dataset is small enough to run on a local environment which is necessary in the context of using local models.

## 1.2 Harmful Retrieval Poisoning Threat Model

The main risk comes from retrieval poisoning. An attacker won't need to affect the model itself, he can simply corrupt the indexed data so the retriever and ranker push the attackers intended result

This risk is in fact very well documented in recommender security research: factorization-based systems [5], graph-based systems [6], top-$N$ ranking [7], and deep recommendation models [8]. RAG system research also report similar issues: poisonned knowledge can affect what the system generates and how it behaves [9, 10]. A specific recommender system research focuse on RAG poisonning also shows poisoned retrieval context can manipulate the output [11].

To get a complete picture we will thus take a look at three type of attacks that are planned to be available in the local codebase: targeted promotion, prompt injection, and untargeted degradation.

## 1.3 RAG-Poison-Lab System Workflow

RAG Poison Lab contains the whole workflow and is made from scratch to cater to our specific use case. It prepares the data, creates the baseline and poisoned files the indexes them on Elasticsearch, shares recommendation and the API routes and traces, and finally keeps artifacts from every run with a specific run ID [1]. We are eseentially trying to design 2 flows one where the system is not attacked and one where it is and then compare the results from the run through two indices: `movies` and `movies_poisoned`.

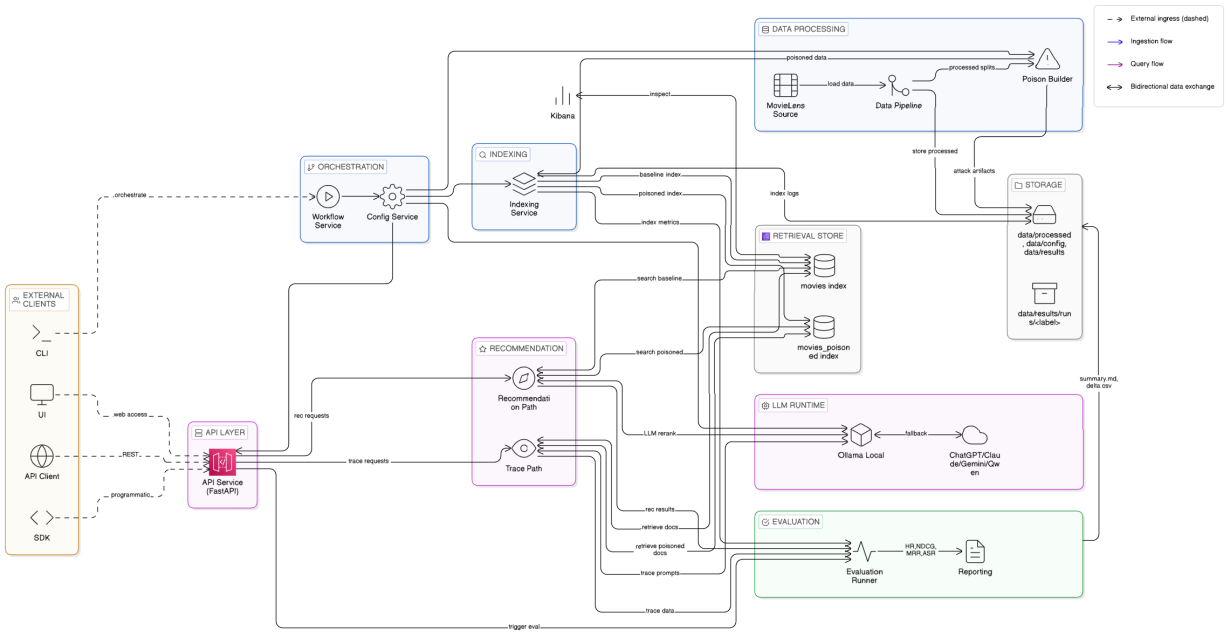


Figure 1-1: Planned architecture figure for RAG-Poison-Lab.

The trace path exposes retrieval query text, retrieved documents, poison markers, and re-ranking internals. This is important for our poisonning evaluation because it shows why the attacks happen and how they work.

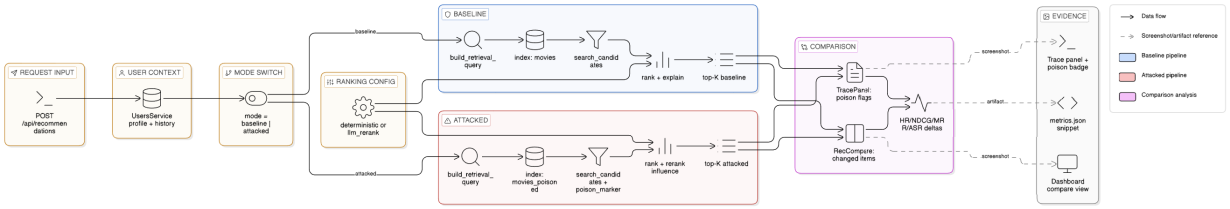


Figure 1-2: Planned baseline-versus-attacked comparison workflow.

In short, the project provides a controlled red team lab where the user request is fixed and only retrieval/index mode changes.

# 2 Related Work

Related studies that support this thesis fall into four groups.

**Recommender poisoning attacks.** Early studies show that a harmful actor could change recommendation results by poisoning user interaction data [5]. Later studies found similar risks for graph recommenders and deep recommenders [6, 8]. Other studies even show that an attacker can achieve targeted manipulation for top-$N$ recommendation [7]. We also notice with PORE that a robustness-based approach for untargeted posisoning can still preserve part of the top-$N$ recommendations even under fake-user injection [12].

**RAG poisoning and security.** Studies like PoisonedRAG show that when retrieval content is corrupted it can change the behavior of the model later on in the RAG pipeline [9, 10], which is why the red team aligns with our proposal. The official USENIX version of poisonedRAG shows that by only using a few injected malicious texts, we can already achieve high attack success, in that case we can see that low-volume poisoning is already practical [13]. While RevPRAG reports through detection research very strong poisoned-response detection with high true-positive and low false-positive results using multiple RAG setups [14].

**Recommendation-specific RAG poisoning.** Poison-RAG studies retrieval poisoning in a recommender system showing that it can cause heavy output shift in the final response[11]. This is close to the problem we are trying to target with RAG Poison Lab. Poison-RAG also highlights that modifying local metadata can be even more effective than global ones in black-box experiments, which supports using different poisoning methods and granularities in our setup [11]. Other research articles like EMNLP on LLM recommenders, show that the order of interactions and re-ranking through LLMs can also affect the recommendation system greatly [15].

**Evaluation metrics for ranked recommendation.** HR, MRR, and NDCG are valid and strong metrics that are used in many studies that aim to measure RAG poisoning, which fits the same goal we are trying to achieve [16, 17].

Looking at the previous mentioned studies, we can see that this proposal offers a system that links attack generation, index-level A/B switching, trace inspection, and report export in one single system that can cater to answering all our questions on the subject.

# 3 Technical Route

## 3.1 Evaluation Metrics

We evaluate by comparing baseline and attacked runs following these conditions: the same users, same $K$, same ranking system (simple or LLM), and the same processed dataset snapshot. We are using the following metrics: HR@K, NDCG@K, and MRR@K, which are standard for recommender and ranking analysis [16, 17], They will show us how the attack affects the quality of the recommendation output.

To better measure and get a greater understanding of the attacks impact, we will also report ASR@K when targeted promotion is active, candidate overlap at $K$, and target-rank lift. These extra metrics make manipulation effects easier to interpret than quality metrics alone.

## 3.2 Attack Design and Experimental Methodology

Attack settings are stored in `attack_config.json`, with The main parameters being attack type, poison fraction, and optional target movie ID. We will evaluate using three attack types:

- Targeted promotion,
- Prompt injection payload insertion,
- Untargeted degradation.

The experiment loop order is:

1. Prepare the baseline processed data,
2. Generate poisoned bulk from baseline bulk,
3. Index baseline and attacked bulks separately,
4. Run recommendation requests,
5. Compute metrics and export the final artifacts.

Control settings stay the same while the parameters of the attack change. This makes it much easier to compare normal retrieval with attacked retrieval and observe the effect of this attack.

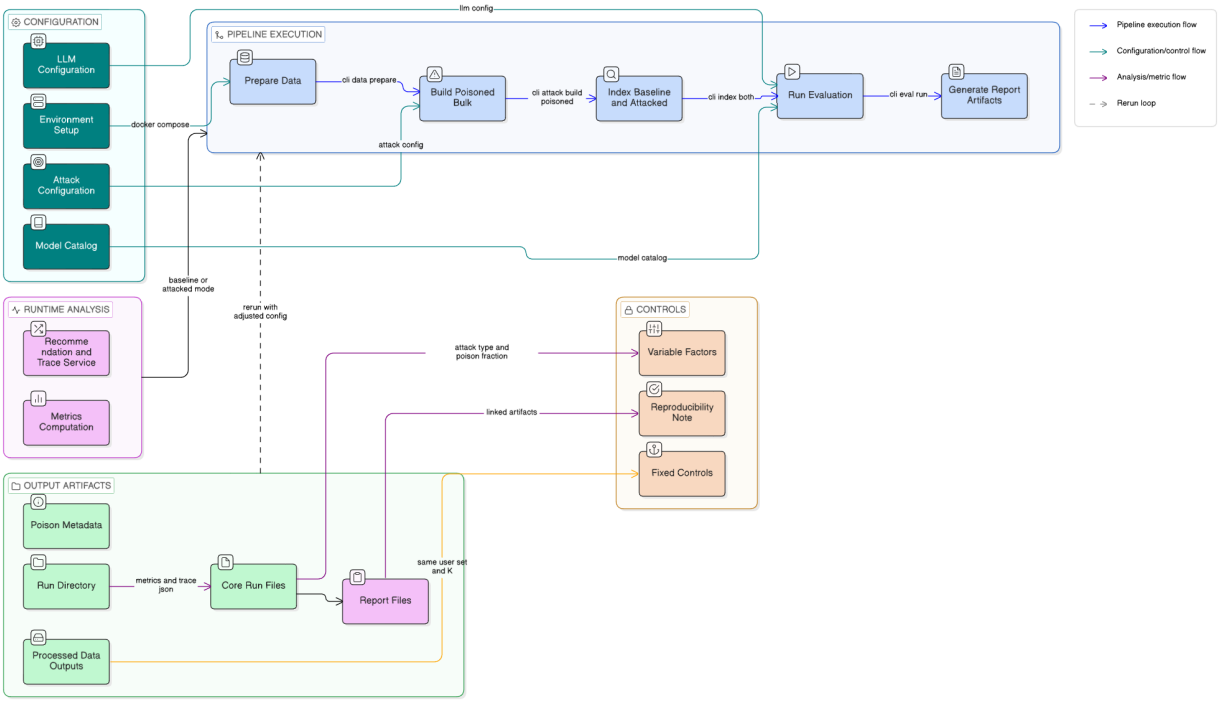


Figure 3-1: Planned experiment pipeline figure for reproducible baseline-vs-attacked evaluation.

# 4 Experimental Environment

## 4.1 Docker Engine and Docker Compose

This thesis does not run one script at a time. It was made efficient by running a stack where each service has a clear role using Docker containers. The Compose file consists of the following services: `Elasticsearch`, `Kibana`, `Ollama`, `RagPoison image containing my system`, and an `indexer` profile for one-shot indexing [1]. The layout is set this way because this recommender system depends on three concurrent states: index state (held by Elasticsearch and Kibana), model state (held by Ollama for local models), and API configuration state (held by my RagPoison image).

The startup order relies on health checks to make sure to avoid any failures when launching containers that depend on each other. The app waits until Elasticsearch and Ollama are reachable, and the indexer waits for Elasticsearch health check before pushing bulk data.

Persistence is also split on purpose:

- Bind mounts for `data/` and `ml-100/`, so generated artifacts stay put even after rebuilds avoiding the need to re-parse the dataset;
- Named volumes for Elasticsearch and Ollama, so local retrieval and model cache stay stable across restarts.

This setup simplifies reproducibility across different environments with the same efficiency and avoids repeating the same task at every built image.

## 4.2 Python Backend and CLI Workflow

The backend API is organized by experiment function. Routers will cover health checks, user/profile access, recommendation requests, trace requests, and LLM settings [1].

A recommendation request in this project follows a fixed backend flow:

- Load user profile and history;
- Build retrieval query text from top genres and liked titles;
- Choose index by mode (`baseline` → `movies`, `attacked` → `movies_poisoned`);
- Retrieve candidates from Elasticsearch while filtering seen movie IDs;
- Rank candidates in deterministic mode or `llm_rerank` mode depending on user choice;
- Return recommendation items plus show debug results.

The `trace` endpoint uses the same mode-to-index logic. This is meant to achieve a fair comparison since baseline and attacked traces are generated with the same request shape. Thanks to this logic, differences will have a higher chance of being linked to changes coming from retrieval changes rather than being caused API-side issues.

A CLI wizard is also made available for the core extensive testing. It maps all the experiment stages such as `data`, `attack`, `index`, `eval`, and `report`. This would be where most of the testing will happen since a CLI interface is more efficient for bulk testing.

## 4.3 TypeScript Frontend

The frontend is based on react and Node.js. It is used to select users, compare baseline vs attacked outputs, view traces, edit runtime settings and its purpose is mainly for displaying demos and visualizing the result of our experiments, while the main logic stay in the backend.

## 4.4 Elasticsearch and Kibana

Elasticsearch is an open-source search engine built on Lucene, and it stores documents in JSON indices and is based on a inverted-index retrieval method [18]. In our project ES will be used as our principal retrieval system and the backbone of this project.

We are using Elasticsearch in the context of poisoning and red teaming simply because this project studies harmful retrieval poisoning. If we change document content at the retrieval level then the candidate set can change which will cause our top-$k$ recommendations to be affected. Naturally in our project having access to that retrieval layer is necessary and Elasticsearch gives us the opportunity to temper with it. This tool is used worldwide so it adds a layer of reality to our simulated environment

In the current implementation, retrieval uses a weighted `multi_match` query over:

- `title3`,
- `genres2`,
- `synopsis`.

The system also applies a `must_not` filter to remove movie IDs already seen by the user so we can receive better recommendations instead of repeating old items. Both baselines index and attacked index use BM25 similarity in mapping settings, which is the default lexical ranking for scored text retrieval in Elasticsearch [18].

Two index names are defined in code:

- `movies` for baseline retrieval,
- `movies_poisoned` for attacked retrieval.

The most important part about the indexing in RAG Poison Lab is the index splitting. This feature allows for A/B comparison without changing endpoints.

The mapping schema will include basic recommendation fields like `movie_id`, `title`, `genres`, `synopsis` and poisoning evidence fields like `poison_marker`, `poison_payload`. Having it present fields this will help with understanding and visualizing attack evidence, it is also included with indexed documents, which will make auditing simpler.

Kibana is the web interface for Elasticsearch [19]. While not being used directly in the ranking or text generation, It will be used for verification before and after runs:

- Check that both indices exist and are healthy,
- Inspect document counts for `movies` and `movies_poisoned`,
- Sample documents in poisoning markers/payload fields for verification,
- Run quick ad hoc queries in Dev Tools when debugging retrieval behavior.

They thus work hand-in-hand, ES will handle retrieval while Kibana makes inspection and verification easier

## 4.5 Optional LLM Runtime and re-ranking

Since the purpose of thesis is to evaluate retrieval poisoning as a whole RAG Poison Lab supports two modes to verify how the poisoning behaves with and without language model re-ranking.

`deterministic` mode uses local ranking logic using retrieval score and preference overlap and is the essential baseline for this project. It doesn't rely on LLMs for ranking.

`llm_rerank` mode builds uses a re-rank prompt based on the retrieved candidates and user preferences. It will then send it to the configured victim model, which parses the candidate order that was received. It then follows up by mapping that order to movie IDs again [1]. This mode is quite important because showing how poisoned prompts can influence the LLM to return hostile re-ranked order that goes along the attackers request rather than innocent results.

The local default runtime is Ollama for current development. This allows for cheap development cost currently before diving into cloud-based solutions when it comes the time to start getting strong metrics. The backend operates with fallbacks: if re-ranking fails, returns invalid JSON, or model access is unavailable, the service falls back to deterministic ranking and records fallback signals for later analysis.

### 4.6 Dataset, Configuration, Outputs, and Reproducibility

MovieLens 100K is used as the benchmark dataset [4]. The preprocessing writes deterministic outputs into `data/processed/` which includes user profiles, train/test splits, and Elasticsearch files.

Configuration is purposefully divided by purpose into two JSON files:

- `attack_config.json`: attack type, poison fraction, target movie, and payload-related settings;
- `llm_config.json`: victim/attacker providers, models, and ranking mode.

We Evaluate outputs by groups via run label in `data/results/runs/<label>/`. We store aggregate values, per-user values, and run metadata including attack context in `metrics.json`, and we store single user runs debug data in `attack_trace.json`. Finally, Reports are produced such as `summary.md`, `delta.csv`, and snapshots of both config files.

An import aspect of our project is seamless reproducibility. The system will save SHA-256 hashes the source and attack config. If there's a mismatch in hash we immediately rebuild the poisoned bulk. This helps make sure all runs are consistent and results are trustworthy.

Overall, we make sure our environment is designed in a way that makes all results collected trusted, and the solution to that is to have every part of every step to be documented, from index state and concrete config files to concrete run artifacts.

## 5 Environmental Sustainability and Project Budget

### 5.1 Environmental Sustainability

RAG-Poison-Lab is designed to be run in a modest experiment environment. This directly reduces energy and cloud usage compared with large remote training workflows.

The project is environmentally friendly for the following reasons:

- **Local-first execution:** core services (backend, Elasticsearch, indexing, evaluation) all run on one local machine with Docker, so experiments do not depend on cloud based data-center infrastructure.
- **Selective LLM usage:** most debugging and baseline checks can be tested in deterministic ranking mode or using a local model; paid API models are used only for deeper and meaningful results later one and are not an active part of our development.
- **Reused artifacts:** We process the dataset *once* and the output is reused throughout the experiment. No need for repeated data preparation.
- **Hash-based refresh control:** Using our hashing method, the only reason for which the poisoned bulk would have to be rebuilt is if the source bulk changes or our attack configuration changes, which speeds up workflow and avoids being forced to re-index and at every run start.
- **Staged evaluation:** `single`, `batch`, and `full` modes allow to quickly single out errors instead of having to use expensive ressources waiting for a full batch run. It also allows for simple demos to be shared.

### 5.2 Project Budget Estimate

Table 1 shows an estimate of the budget for this project. API costs are for a capped amount of credits and could vary depending on the benchmarking token use. The objective is to not go past these token estimates by oiptimizing prompts to use the least amount of token possible without affecting results.

Table 1: Estimated one-semester budget for RAG-Poison-Lab

| Item | Basis of estimate | Cost |
|---|---|---|
| Local electricity | Modest laptop/workstation with a minimum of 8 cpus and 16gb of ram (ideally 12 cpus and 32gb of ram) in regular daily runs across one semester | 400 |
| Storage and backup | External storage/cloud backup for data, metrics, traces, and reports | 500 |
| OpenAI API credit | Controlled comparison runs with token cap | 800 |
| Claude API credit | Controlled comparison runs with token cap | 1000 |
| Gemini API credit | Controlled comparison runs with token cap | 800 |
| Qwen API credit | Controlled comparison runs with token cap | 400 |
| Miscellaneous buffer | Extra reruns, troubleshooting, small unexpected costs | 250 |
| **Total** | | **4150 RMB** |

## 6 Schedule

The schedule is planned for Spring 2026 with explicit checkpoints and one revision buffer.

| Period | Planned work |
|---|---|
| March 2026 | Verify reproducibility, and run pilot single-user red team tests. |
| April 2026 | Run batch/full evaluations for all attack types under deterministic and LLM re-rank modes. |
| May 2026 | Analyze metric and trace differences, produce final figures/tables, and complete thesis writing. |
| Early June 2026 | Supervisor revision cycle, reproducibility re-check, and final submission preparation. |

Risk plan: If full-scale runs are delayed, batch checkpoints will be prioritized first, then expanded after stability checks.

# References


[1] Sbaaoui Idriss. Rag-poison-lab (thesis repository). https://github.com/6ba3i/thesis, 2026. Accessed: 2026-03-08.

[2] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Kuttler, Mike Lewis, Wen-tau Yih, Tim Rocktaschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. *arXiv preprint arXiv:2005.11401*, 2020.

[3] Yupeng Hou, Junjie Zhang, Zihan Lin, Hongyu Lu, Ruobing Xie, Julian McAuley, and Wayne Xin Zhao. Large language models are zero-shot rankers for recommender systems. *arXiv preprint arXiv:2305.08845*, 2023.

[4] F. Maxwell Harper and Joseph A. Konstan. The movielens datasets. *ACM Transactions on Interactive Intelligent Systems*, 5(4):1–19, 2015.

[5] Bo Li, Yining Wang, Aarti Singh, and Yevgeniy Vorobeychik. Data poisoning attacks on factorization-based collaborative filtering. In *Advances in Neural Information Processing Systems*, volume 29. Curran Associates, Inc., 2016.

[6] Minghong Fang, Guolei Yang, Neil Zhenqiang Gong, and Jia Liu. Poisoning attacks to graph-based recommender systems. *arXiv preprint arXiv:1809.04127*, 2018.

[7] Minghong Fang, Neil Zhenqiang Gong, and Jia Liu. Influence function based data poisoning attacks to top-n recommender systems. *arXiv preprint arXiv:2002.08025*, 2020.

[8] Hai Huang, Jiaming Mu, Neil Zhenqiang Gong, Qi Li, Bin Liu, and Mingwei Xu. Data poisoning attacks to deep learning based recommender systems. *arXiv preprint arXiv:2101.02644*, 2021.

[9] Wei Zou, Runpeng Geng, Binghui Wang, and Jinyuan Jia. Poisonedrag: Knowledge corruption attacks to retrieval-augmented generation of large language models. *arXiv preprint arXiv:2402.07867*, 2024.

[10] Baolei Zhang, Yuxi Chen, Zhuqing Liu, Lihai Nie, Tong Li, Zheli Liu, and Minghong Fang. Practical poisoning attacks against retrieval-augmented generation. *arXiv preprint arXiv:2504.03957*, 2025.

[11] Fatemeh Nazary, Yashar Deldjoo, and Tommaso Di Noia. Poison-rag: Adversarial data poisoning attacks on retrieval-augmented generation in recommender systems. *arXiv preprint arXiv:2501.11759*, 2025.

[12] Jinyuan Jia, Yupei Liu, Yuepeng Hu, and Neil Zhenqiang Gong. PORE: Provably robust recommender systems against data poisoning attacks. In *32nd USENIX Security Symposium (USENIX Security 23)*, pages 1703–1720. USENIX Association, 2023.

[13] Wei Zou, Runpeng Geng, Binghui Wang, and Jinyuan Jia. Poisonedrag: Knowledge corruption attacks to retrieval-augmented generation of large language models. In *34th USENIX Security Symposium (USENIX Security 25)*, pages 3827–3844. USENIX Association, 2025.

[14] Xue Tan, Hao Luan, Mingyu Luo, Xiaoyan Sun, Ping Chen, and Jun Dai. Revprag: Revealing poisoning attacks in retrieval-augmented generation through LLM activation analysis. In *Findings of the Association for Computational Linguistics: EMNLP 2025*, pages 12999–13011. Association for Computational Linguistics, 2025.

[15] Zihan Wang, Zhongxing Cheng, Ming Gao, Wenqiang Gao, Jing Bian, Xiang Liu, and Xin Chen. Enhancing high-order interaction awareness in LLM-based recommender model. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pages 11696–11711. Association for Computational Linguistics, 2024.

[16] Jonathan L. Herlocker, Joseph A. Konstan, Loren G. Terveen, and John T. Riedl. Evaluating collaborative filtering recommender systems. *ACM Transactions on Information Systems*, 22(1):5–53, 2004.

[17] Yining Wang, Liwei Wang, Yuanzhi Li, Di He, and Tie-Yan Liu. A theoretical analysis of NDCG type ranking measures. In *Proceedings of the 26th Annual Conference on Learning Theory*, volume 30 of *Proceedings of Machine Learning Research*, pages 25–54, 2013.

[18] Elastic. Elasticsearch: The search and analytics engine. https://www.elastic.co/elasticsearch, 2026. Accessed: 2026-03-08.

[19] Elastic. Kibana: Explore, visualize, and manage data in elasticsearch. https://www.elastic.co/kibana, 2026. Accessed: 2026-03-08.